

Structure du Dépôt — CareWay

Pour développeurs | Version : 0.1.0 | Février 2026

Stack : React 18 + TypeScript + Vite + Supabase + Tailwind CSS

Table des matières

- 1. Arborescence synthétique
- 2. Rôle de chaque dossier
- 3. Pages et composants
- 4. Services API et client Supabase
- 5. Routing et state management
- 6. Styles et theming
- 7. Lancer en développement
- 8. Bonnes pratiques et conventions

1. Arborescence synthétique

CareWay/	
├─ docs/	# Documentation (USER_GUIDE, DATA_MODEL, etc.)
├─ db/	
│ └─ schema.sql	# Schéma SQL complet – à exécuter dans Supabase
├─ docker/	
│ ├── Dockerfile	# Multi-stage : dev / build / production (nginx)
│ ├── docker-compose.yml	# Stack complète : frontend + Supabase local
│ ├── nginx.conf	# Config nginx production
│ ├── nginx.workflows.conf	# Config nginx pour workflows
│ └─ .env	# Variables Docker (JWT_SECRET, ANON_KEY...)
├─ olga/	# Config Olga designer/admin (sous-projet
séparé)	
│ ├── Dockerfile.admin	
│ ├── Dockerfile.designer	
│ └─ config/	
├─ public/	# Fichiers statiques (favicon, images...)
├─ src/	
│ └─ App.tsx	# Composant racine – routing par état, contexte
└─ lang	
│ ├── main.tsx	# Point d'entrée React (ReactDOM.createRoot)
│ ├── index.css	# CSS global
│ └─ components/	# Tous les composants React
│ ├── ui/	# Composants UI génériques (shadcn/ui + Radix)
│ ├── figma/	# Composants issus du design Figma
│ ├── HomePage.tsx	# Page d'accueil (sélection rôle)
│ ├── ModernLoginForm.tsx	# Connexion patient
│ ├── ModernRegisterForm.tsx	# Inscription patient
│ └─ FranceConnectLogin.tsx	# Auth infirmier/médecin via FranceConnect

```

├── Dashboard.tsx          # Dashboard générique (wrapper)
├── PatientDashboard.tsx   # Dashboard patient complet
├── NurseDashboard.tsx     # Dashboard infirmier + médecin
├── MedecinDashboard.tsx   # Logique métier médecin
├── AppointmentScheduler.tsx # Formulaire de réservation patient
├── AppointmentDetailsModal.tsx # Détail d'un RDV
├── AppointmentConfirmDialog.tsx # Confirmation d'action
├── BookingPage.tsx        # Page de réservation complète
├── AIRouteOptimizer.tsx   # Optimisation tournée avec IA
├── InteractiveMap.tsx     # Carte Leaflet interactive
├── NurseResultsPage.tsx   # Résultats de recherche d'infirmiers
├── NurseSettings.tsx      # Paramètres professionnels
├── CareTypeSelector.tsx   # Sélecteur de type de soin
├── PatientProfile.tsx     # Profil médical patient
├── PatientProfileContent.tsx # Contenu du profil
├── DocumentUploadModal.tsx # Upload de documents médicaux
├── UnavailabilityManager.tsx # Gestion des indisponibilités
├── VisitRecapModal.tsx    # Récapitulatif visite
├── MedecinValidationsModal.tsx # Validation prescriptions médecin
├── DatabaseDebug.tsx      # Outil de debug DB (dev uniquement)
├── LanguageContext.tsx    # Context React pour i18n
├── LanguageSwitcher.tsx   # Bouton FR/EN
├── translations.ts        # Dictionnaire de traductions
├── workflow/              # (dossier vide – fonctionnalité à venir)

├── services/
├──   └── api.tsx          # Toutes les fonctions appel API (fetch vers
Edge Fn)

├── constants/
├──   └── careTypes.ts     # Liste statique des types de soins

├── utils/
├──   ├── supabase/
├──   │   └── info.tsx     # projectId + publicAnonKey (config Supabase)
├──   └── testWorkflow.ts  # Script de test workflow (dev)

├── stores/                # (vide – Zustand/Context prévu)
├── styles/                # Styles additionnels (globals.css)
├── supabase/              # Config Supabase locale
├── guidelines/            # Guidelines du projet (Markdown)

├── supabase/
├──   └── functions/        # Edge Functions Supabase (backend serverless)
├── volumes/               # Volumes Docker Supabase local
├──   ├── api/
├──   ├── db/
├──   └── storage/

├── index.html             # Template HTML Vite
├── package.json           # Dépendances et scripts npm
├── vite.config.ts         # Config Vite (aliases, plugin react-swc)
├── tsconfig.json          # Config TypeScript
├── tsconfig.node.json     # Config TypeScript pour Node (Vite)
├── README.md              # Documentation principale

```

2. Rôle de chaque dossier

Dossier	Rôle
src/components/	Tous les composants React de l'application
src/components/ui/	Composants UI réutilisables générés par shadcn/ui (Button, Dialog, Input...)
src/components/figma/	Composants fidèles au design Figma SAE#16
src/services/	Couche d'abstraction pour tous les appels HTTP/API
src/utils/supabase/	Credentials Supabase (projectId, publicAnonKey)
src/constants/	Données statiques de l'application (types de soins...)
src/stores/	State global (vide actuellement — à utiliser pour Zustand ou Context)
src/styles/	CSS global additionnel (au-delà de Tailwind)
db/	Script SQL de création de la base de données
docker/	Orchestration Docker (compose + Dockerfile + nginx)
supabase/functions/	Edge Functions Supabase (backend serverless TypeScript/Deno)
public/	Assets statiques (non transformés par Vite)

3. Pages et composants

CareWay n'utilise **pas de React Router**. La navigation est gérée par **état local** dans `App.tsx` via `useState<'home' | 'patientAuth' | 'nurseAuth' | 'dashboard'>`.

Carte des vues

```
App.tsx
├── home      → <HomePage>
├── patientAuth
│   ├── login → <ModernLoginForm>
│   └── register → <ModernRegisterForm>
├── nurseAuth → <FranceConnectLogin>
└── dashboard
    ├── type=patient → <PatientDashboard>
    └── type=nurse|medecin → <NurseDashboard>
```

Composants réutilisables clés

Composant	Fichier	Props principales
-----------	---------	-------------------

Composant	Fichier	Props principales
AppointmentScheduler	components/AppointmentScheduler.tsx	patientId, onSuccess
AIRouteOptimizer	components/AIRouteOptimizer.tsx	infirmierId, date
InteractiveMap	components/InteractiveMap.tsx	nurses, selectedNurse
DocumentUploadModal	components/DocumentUploadModal.tsx	patientId, onClose
UnavailabilityManager	components/UnavailabilityManager.tsx	infirmierId
LanguageSwitcher	components/LanguageSwitcher.tsx	(via Context)

4. Services API et client Supabase

Architecture API

L'application communique avec le backend exclusivement via des **Supabase Edge Functions** exposées sur une URL du type :

```
https://<projectId>.supabase.co/functions/v1/make-server-1b83ce4c/<endpoint>
```

Toutes les fonctions sont centralisées dans :

```
src/services/api.tsx
```

Chaque appel utilise `fetch()` natif avec le header `Authorization: Bearer <publicAnonKey>`.

Fonctions disponibles (extrait)

Fonction	Méthode	Endpoint
<code>getCareTypes()</code>	GET	<code>/care-types</code>
<code>getPatientAppointments(id)</code>	GET	<code>/appointments/patient/:id</code>
<code>getInfirmierAppointments(id)</code>	GET	<code>/appointments/infirmier/:id</code>
<code>getPendingAppointments(id?)</code>	GET	<code>/appointments/pending/:id</code> ou <code>/all</code>
<code>updateAppointment(id, updates)</code>	PATCH	<code>/appointments/:id</code>
<code>acceptAppointment(id, nurseId)</code>	PATCH	<code>/appointments/:id</code> → <code>status=confirmed</code>
<code>getPatientProfile(id)</code>	GET	<code>/api/patient/:id/profile</code>
<code>updatePatientProfile(id, data)</code>	PATCH	<code>/api/patient/:id/profile</code>

Config Supabase

```
// src/utils/supabase/info.tsx
export const projectId = '...'; // ID du projet Supabase
export const publicAnonKey = '...'; // Clé anon publique
```

⚠ En mode Docker local, ces valeurs doivent pointer vers `localhost:8000` avec la clé anon locale.
Voir la section `.env.development` dans le README.

5. Routing et State Management

Routing

- **Pas de React Router DOM** — navigation par `useState` dans `App.tsx`
- Ajouter React Router si le projet évolue : `npm install react-router-dom`

State Management

- **Pas de store global** (Zustand, Redux) — `src/stores/` est vide
- L'état utilisateur (`currentUser`) est maintenu dans `App.tsx` et passé en props
- **Context React** uniquement pour l'internationalisation (`LanguageContext`)

Internationalisation (i18n)

```
// src/components/LanguageContext.tsx – Context provider
// src/components/translations.ts – Dictionnaire FR/EN
// type Language = 'fr' | 'en'
```

Utilisation dans un composant :

```
const { t, language } = useContext(LanguageContext);
const label = t('myKey'); // retourne la traduction
```

6. Styles et Theming

- **Tailwind CSS** : classes utilitaires (configuré dans `tailwind.config.js`)
- **shadcn/ui** : composants accessibles basés sur Radix UI, dans `src/components/ui/`
- **CSS global** : `src/index.css` et `src/styles/globals.css`
- **Thème** : `next-themes` est installé — dark mode disponible mais non activé par défaut

7. Lancer en développement

Mode local (sans Docker)

```
# 1. Installer les dépendances
npm install

# 2. Configurer les variables d'environnement
# Créer .env à la racine :
# VITE_SUPABASE_URL=https://<project-id>.supabase.co
# VITE_SUPABASE_ANON_KEY=<anon-key>

# 3. Lancer le serveur de dev
npm run dev
# → http://localhost:5173
```

Mode Docker (Supabase local inclus)

```
# Stack complète (frontend + Supabase local)
npm run docker:start
# → Frontend : http://localhost:3000
# → Studio   : http://localhost:54323

# Arrêter
npm run docker:stop

# Nettoyer (supprime les volumes)
npm run docker:clean
```

8. Bonnes pratiques et conventions

Naming

Type	Convention	Exemple
Composant React	PascalCase	PatientDashboard.tsx
Fichier utilitaire	camelCase	api.tsx, testWorkflow.ts
Constante	camelCase	careTypes.ts
Type/Interface TypeScript	PascalCase	interface Appointment {}

Où ajouter un composant ?

```
src/components/MonNouveauComposant.tsx
```

- Si c'est un composant UI générique (bouton, modal...) → `src/components/ui/`
- Si c'est lié au design Figma → `src/components/figma/`
- Si c'est une page entière → `src/components/` + déclarer dans `App.tsx`

Où ajouter un appel API ?

→ `src/services/api.tsx` — ajouter une fonction `export const maFonction = async (...) => { ... }`

Où ajouter une constante ?

→ `src/constants/` — créer un fichier si nécessaire ou étendre `careTypes.ts`

Où ajouter du state global ?

→ `src/stores/` — créer un store Zustand ou un nouveau Context React

Ajouter une traduction

Éditer `src/components/translations.ts` :

```
export const translations: Record<Language, Record<string, string>> = {  
  fr: { maClé: 'Mon texte en français' },  
  en: { maClé: 'My text in English' },  
};
```

TypeScript strict

Le projet utilise `tsconfig.json` avec les options strictes activées. Toujours typer les props des composants et les retours de fonctions API.

Document généré pour le projet SAE#16 — NEXUS AI Innovation Lab for Health